High-Level Design (HLD)

“RAG-Based College/University FAQ Assistant using LangGraph with Human-in-the-Loop (HITL)”

Problem Statement

Customer support systems often rely on human agents to answer repetitive queries, leading to increased response time and operational cost. In academic institutions, students frequently ask questions regarding attendance, exams, grading, and fees.

There is a need for an intelligent system that can automatically answer user queries based on institutional documents while ensuring accuracy and providing escalation to human support when required.

Objective

The objective of this project is to design and develop a Retrieval-Augmented Generation (RAG) based customer support assistant that can:

- Process a PDF knowledge base
- Retrieve relevant information using embeddings
- Answer queries contextually using an LLM
- Use LangGraph for workflow orchestration
- Perform conditional routing based on query intent
- Enable Human-in-the-Loop (HITL) escalation for complex queries

Scope of the System

The system will:

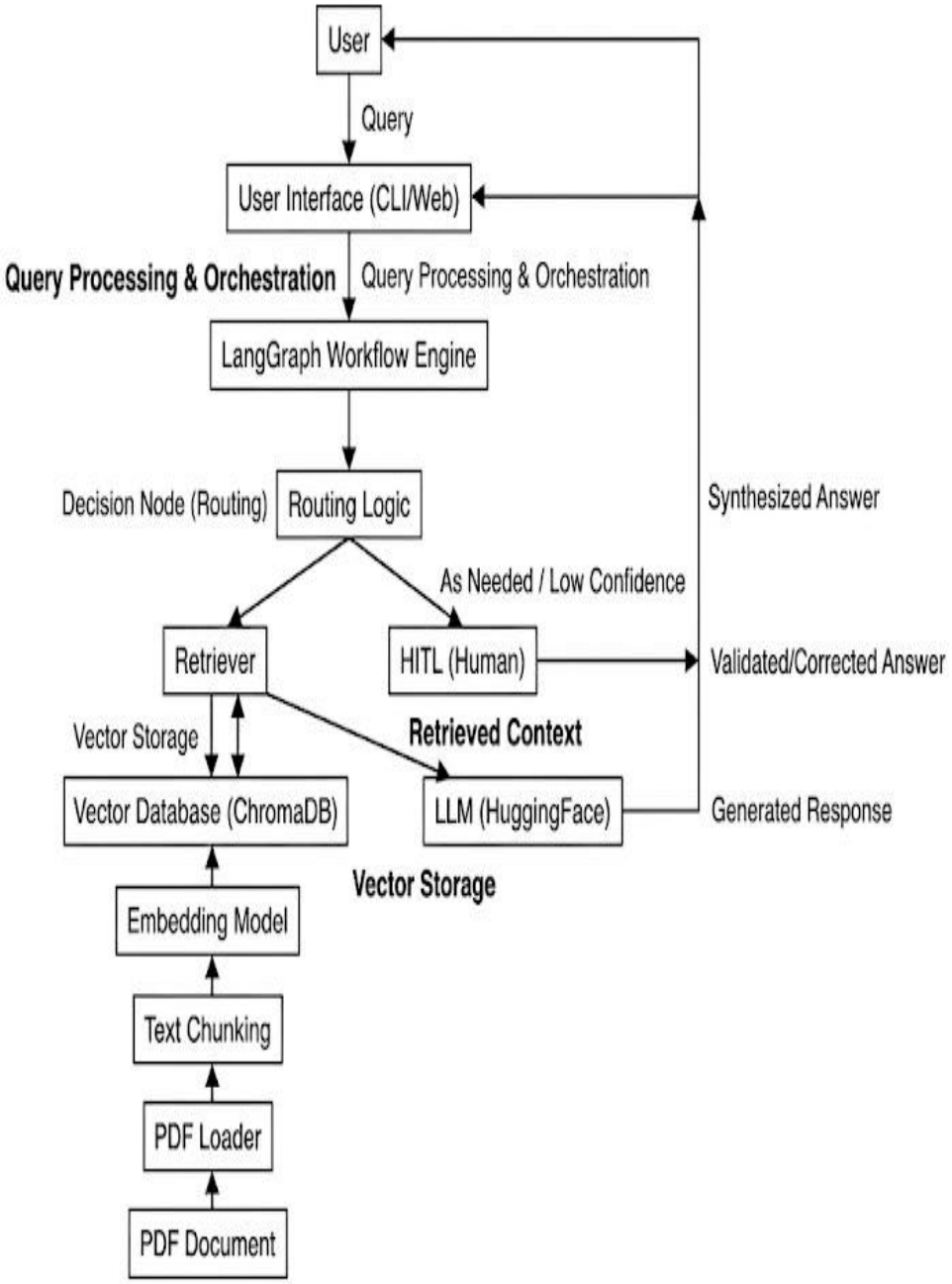
- Accept user queries through a simple interface (CLI/Web)
- Process and store a PDF document as a knowledge base
- Retrieve relevant content using vector search (ChromaDB)
- Generate answers using a language model
- Use a graph-based workflow for processing queries
- Escalate queries to human support when confidence is low

The system will not include:

- User authentication
- Multi-user scaling (initial version)
- Real-time human agent integration (simulated HITL only)

High-Level Design (HLD)

Architecture Diagram



Component Description

1. User Interface (CLI/Web)
   The user interface allows users to input queries and receive responses. It acts as the entry point of the system and can be implemented as a command-line interface or a web-based interface.
2. Document Loader
   The document loader is responsible for loading the PDF knowledge base into the system. It extracts textual data from the PDF document for further processing.
3. Text Chunking Module
   This module splits the extracted text into smaller chunks to improve retrieval efficiency. Proper chunking ensures better context matching during query processing.
4. Embedding Model
   The embedding model converts text chunks into numerical vector representations. These embeddings capture the semantic meaning of the text and are used for similarity search.
5. Vector Database (ChromaDB)
   ChromaDB stores the embeddings generated from text chunks. It enables efficient similarity-based retrieval of relevant information during query processing.
6. Retriever
   The retriever searches the vector database to find the most relevant chunks based on the user query. It plays a crucial role in retrieving context for the language model.
7. Large Language Model (LLM)
   The LLM generates responses using the retrieved context. It processes the user query along with relevant information to produce accurate and meaningful answers.
8. LangGraph Workflow Engine
   LangGraph manages the overall workflow of the system. It defines the flow of execution between different components and ensures proper orchestration of tasks.
9. Routing Logic
   The routing logic acts as a decision node that determines whether the query should be handled automatically or escalated to a human. It is based on factors such as confidence level and query complexity.

10. Human-in-the-Loop (HITL) Module
    The HITL module enables human intervention when the system is unable to provide a confident answer. It ensures reliability and improves user trust in the system.

Data Flow

The data flow of the system describes how information moves from the input document to the final response generated for the user.

1. Document Ingestion
   The system begins by loading the PDF document using the document loader. The textual content is extracted from the PDF.
2. Text Chunking
   The extracted text is divided into smaller chunks using a chunking strategy. This ensures efficient retrieval and better context handling.
3. Embedding Generation
   Each text chunk is converted into a vector representation using an embedding model. These embeddings capture the semantic meaning of the text.
4. Vector Storage
   The generated embeddings are stored in a vector database (ChromaDB). This allows fast similarity-based retrieval.
5. User Query Input
   The user submits a query through the user interface.
6. Query Processing
   The query is processed by the LangGraph workflow engine, which manages the execution flow.
7. Retrieval
   The retriever searches the vector database to find the most relevant chunks based on the query.
8. Context Passing to LLM
   The retrieved chunks are passed as context to the Large Language Model (LLM).
9. Response Generation
   The LLM generates a response using both the user query and the retrieved context.

10. Conditional Routing
    The routing logic evaluates the response. If the confidence is low or the query is complex, it is routed to the Human-in-the-Loop (HITL) module.
11. Human Intervention (if required)
    A human reviews the query and provides a corrected or validated response.
12. Final Output
    The final response (either generated or human-validated) is sent back to the user through the interface.

Technology Choices

1. ChromaDB
   ChromaDB is used as the vector database for storing embeddings. It is lightweight, easy to integrate, and efficient for similarity search operations.
2. LangGraph
   LangGraph is used to design and manage the workflow of the system. It enables graph-based orchestration, making it easier to implement conditional routing and control logic.
3. Embedding Model
   A HuggingFace-based embedding model is used to convert text into vector representations. It provides good semantic understanding and is suitable for RAG applications.
4. Large Language Model (LLM)
   A HuggingFace LLM is used to generate responses. It allows flexible deployment without relying on paid APIs and integrates well with the RAG pipeline.
5. Python
   Python is used as the primary programming language due to its rich ecosystem of libraries for NLP, machine learning, and AI development.
6. PDF Processing Library
   Libraries such as PyPDF are used to extract text from PDF documents efficiently.
7. Frontend Interface
   A simple CLI or web interface is used to interact with the system, allowing users to input queries and receive responses.

Scalability Considerations

1. Handling Large Documents
   For large PDF files, efficient chunking strategies can be used to ensure optimal retrieval performance without overloading the system.
2. Increasing Query Load
   To handle multiple user queries, the system can be extended with parallel processing and caching mechanisms.
3. Latency Optimization
   Response time can be improved by optimizing retrieval operations and using efficient embedding models.
4. Storage Optimization
   Vector databases can be scaled horizontally to handle large volumes of embeddings.
5. Model Optimization
   Lightweight or quantized models can be used to reduce computational overhead and improve performance.
6. Future Deployment
   The system can be deployed on cloud platforms such as AWS or Firebase to support real-world usage.